

Java™ Management Extensions White Paper

Dynamic Management for the Service Age



THE NETWORK IS THE COMPUTER™

A Sun Microsystems, Inc. Business
901 San Antonio Road
Palo Alto, CA 94303 USA
650 960-1300 fax 650 969-9131

Revision 01, June 1999

This product or document is protected by copyright and distributed under licenses restricting its use, copying, distribution, and decompilation. No part of this product or document may be reproduced in any form by any means without prior written authorization of Sun and its licensors, if any. Third-party software, including font technology, is copyrighted and licensed from Sun suppliers.

Parts of the product may be derived from Berkeley BSD systems, licensed from the University of California. UNIX is a registered trademark in the U.S. and other countries, exclusively licensed through X/Open Company, Ltd.

Sun, Sun Microsystems, the Sun logo, The Network Is The Computer, Solaris, Java, Java Dynamic Management, JavaBeans, Jini, and Javadoc are trademarks, registered trademarks, or service marks of Sun Microsystems, Inc. in the U.S. and other countries. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. in the U.S. and other countries. Products bearing SPARC trademarks are based upon an architecture developed by Sun Microsystems, Inc.

The OPEN LOOK and Sun™ Graphical User Interface was developed by Sun Microsystems, Inc. for its users and licensees. Sun acknowledges the pioneering efforts of Xerox in researching and developing the concept of visual or graphical user interfaces for the computer industry. Sun holds a non-exclusive license from Xerox to the Xerox Graphical User Interface, which license also covers Sun's licensees who implement OPEN LOOK GUIs and otherwise comply with Sun's written license agreements.

RESTRICTED RIGHTS: Use, duplication, or disclosure by the U.S. Government is subject to restrictions of FAR 52.227-14(g)(2)(6/87) and FAR 52.227-19(6/87), or DFAR 252.227-7015(b)(6/95) and DFAR 227.7202-3(a).

DOCUMENTATION IS PROVIDED "AS IS" AND ALL EXPRESS OR IMPLIED CONDITIONS, REPRESENTATIONS AND WARRANTIES, INCLUDING ANY IMPLIED WARRANTY OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE OR NON-INFRINGEMENT, ARE DISCLAIMED, EXCEPT TO THE EXTENT THAT SUCH DISCLAIMERS ARE HELD TO BE LEGALLY INVALID.

Copyright 1997 Sun Microsystems, Inc., 901 San Antonio Road, Palo Alto, Californie 94303 Etats-Unis. Tous droits réservés.

Ce produit ou document est protégé par un copyright et distribué avec des licences qui en restreignent l'utilisation, la copie, la distribution, et la décompilation. Aucune partie de ce produit ou document ne peut être reproduite sous aucune forme, par quelque moyen que ce soit, sans l'autorisation préalable et écrite de Sun et de ses bailleurs de licence, s'il y en a. Le logiciel détenu par des tiers, et qui comprend la technologie relative aux polices de caractères, est protégé par un copyright et licencié par des fournisseurs de Sun.

Des parties de ce produit pourront être dérivées des systèmes Berkeley BSD licenciés par l'Université de Californie. UNIX est une marque déposée aux Etats-Unis et dans d'autres pays et licenciée exclusivement par X/Open Company, Ltd.

Sun, Sun Microsystems, le logo Sun, The Network Is The Computer, Solaris, Java, Java Dynamic Management, JavaBeans, Jini, et Javadoc sont des marques de fabrique ou des marques déposées, ou marques de service, de Sun Microsystems, Inc. aux Etats-Unis et dans d'autres pays. Toutes les marques SPARC sont utilisées sous licence et sont des marques de fabrique ou des marques déposées de SPARC International, Inc. aux Etats-Unis et dans d'autres pays. Les produits portant les marques SPARC sont basés sur une architecture développée par Sun Microsystems, Inc.

L'interface d'utilisation graphique OPEN LOOK et Sun™ a été développée par Sun Microsystems, Inc. pour ses utilisateurs et licenciés. Sun reconnaît les efforts de pionniers de Xerox pour la recherche et le développement du concept des interfaces d'utilisation visuelle ou graphique pour l'industrie de l'informatique. Sun détient une licence non exclusive de Xerox sur l'interface d'utilisation graphique Xerox, cette licence couvrant également les licenciés de Sun qui mettent en place l'interface d'utilisation graphique OPEN LOOK et qui en outre se conforment aux licences écrites de Sun.

CETTE PUBLICATION EST FOURNIE "EN L'ETAT" ET AUCUNE GARANTIE, EXPRESSE OU IMPLICITE, N'EST ACCORDEE, Y COMPRIS DES GARANTIES CONCERNANT LA VALEUR MARCHANDE, L'APTITUDE DE LA PUBLICATION A REpondre A UNE UTILISATION PARTICULIERE, OU LE FAIT QU'ELLE NE SOIT PAS CONTREFAISANTE DE PRODUIT DE TIERS. CE DENI DE GARANTIE NE S'APPLIQUERAIT PAS, DANS LA MESURE OU IL SERAIT TENU JURIDIQUEMENT NUL ET NON AVENU.



Please
Recycle



Adobe PostScript

Java™ Management Extensions White Paper

Dynamic Management for the Service Age

Corporations and service providers are facing new challenges to manage service-driven environments in all industry segments. In these environments, the managed resources (which can indifferently be applications, devices, services or network policies) appear, move and disappear across the network, as they are created, installed, activated and used at a pace never experienced before.

Such challenges can hardly be met by current management technologies of the second generation. These are static in nature and can therefore only work with resources that are known in advance by all the elements of the management system. A static management system is incapable of managing new services since they are now added faster than the system's implementation can be updated.

Java technology offers a great deal, but we would gain little from rewriting existing static management systems in the Java programming language. What is needed are standard, universal extensions of the language that will make any Java technology-based object inherently manageable, the definition of a core set of services for managing these objects, and the means to interface with existing management solutions.

A Charter for Java Management Extensions

Cross Industry Management through Java Technology

Java technology has much more to offer to the developers of system, network and service management solutions. Java technology is dynamic, flexible, and portable: these unique features make it an ideal foundation for building third-generation management solutions. Such solutions are dynamic in nature and can therefore meet the requirements of service driven management over heterogeneous networks and platforms.

Through the Java Community Process, experts from Sun Microsystems, Inc. and leading companies in the management field have developed a specification for doing management *of* and *through* the Java programming language. This process insures that the cumulated experience of the international Java technology community is channeled to produce a universal, open extension of the Java programming language for management.

The Java Management extensions (JMX) have thus been created to keep the Java platform at the leading edge of information technology, to meet market requirements for dynamic management, and to offer the right tools to designers and developers of management solutions. By design, this new standard is suitable for adapting legacy systems, implementing new management solutions and preparing those of the future, while remaining open to applications across all industries. With this charter, JMX is the definitive means for Java management, for all of the Java technology's uses and wherever management is or will be needed.

Who Will Benefit from Java Management?

All developers of Java technology-based applications need a standard way of making Java software manageable. The instrumentation technology that gives manageability needs to be simple and universal. This need is felt across all segments of the industry, from consumer device and storage vendors, to enterprise business applications suppliers and telecommunications equipment manufacturers.

JMX provides a simple, light-weight, standard way to instrument Java objects. JMX instrumentation has very few constraints because it is totally independent from the management infrastructure. This means that a resource can be made manageable

without regards as to how its manager is implemented, for example TMN or SNMP. The device, application or service developer can focus on his core business, rather than investing time and money in the development of management systems.

And because JMX is designed to be a universal management standard, developers only need to instrument their product once to make it manageable everywhere by all other JMX-compliant managers. For example, the same resource or service could be inserted into both TMN and SNMP management environments (and many others) using JMX, without any modification.

Developers of management solutions are faced with rapidly changing technologies as service-driven networking gradually replaces the client-server model. They need applications which will transition from the current management technologies to a fully open model where management services are plugged in dynamically and immediately available.

JMX takes advantage of Java technology's unique features for building a dynamic management solution that is also able to inter-operate with current management systems. JMX frees the management application from a reliance on a fixed information model or communication protocol. New services can be fetched over the network and plugged into the manager as they are deployed.

Because it lets you adapt to any technology, JMX protects application writers and service providers from shifts in management technologies. As new technologies appear, they can be integrated and adopted by the management system.

System integrators, service providers and end-users are strong proponents of open standards, yet paradoxically, they must continually derive from standards in order to differentiate their product from competitors' in the marketplace. Managing similar resources supplied by different vendors that claim conformance to the same standards, but are actually implemented in divergent ways, becomes problematic, at least it is with static management systems.

A dynamic management system however is able to update its view of the system, and to be extended on the fly, whenever required. JMX offers such a new approach: it does not address the semantic of the managed resources, but specifies a management infrastructure into which self-describing components can be added on-the-fly, leaving room for the suppliers' differentiators to be introduced, provisioned and maintained when needed. JMX thus turns the management infrastructure from a necessary evil into a competitive advantage.

Proven Dynamic Management

Using JMX, adding manageability to any Java application is simple. JMX compliant instrumentation can be added to any Java application, sometimes with as little as three to five lines of code. This permits developers, whatever their target market segment, to actually concentrate on their core business, rather than putting excessive

effort into adding manageability. This effectively separates the creation of devices and services from the development of their management systems. JMX defines how you make your product manageable, not how you manage it, thereby freeing your application to use any JMX management application.

New technologies are often regarded with some scepticism in the marketplace. “Perhaps such a big shift in the focus of management will be difficult to implement?” “Maybe we should wait until this new technology is tried and proven?” But JMX provides a dynamic management technology that is easy to implement and integrate. Finally, all the concepts upon which it is based have already been successfully deployed in the Java Dynamic Management™ Kit, a product which has been shipping for the past 18 months.

The Java Management Extensions

JMX defines a management architecture, APIs, and management services all under a single umbrella specification. The specification for JMX has been developed by Sun Microsystems, Inc. along with management industry leaders, following the Java Community Process.

JMX provides developers of Java technology-based applications across all industries with the means to instrument Java platform code, create smart agents and managers in the Java programming language, implement distributed management middle-ware, and smoothly integrate these solutions into existing management systems. The JMX architecture is divided into three levels:

- Instrumentation level
- Agent level
- Manager level

In addition, JMX provides a number of Java APIs for existing standard management protocols. These APIs are independent of the three-level model, yet they are essential because they enable JMX applications in the Java programming language to link with existing management technologies. This part of the JMX specification is referred to as the:

- Additional Management Protocol APIs

JMX Architecture

The JMX architecture is built according to a three-level model. This gives flexibility by allowing subsets of the specification to be used individually by different developer communities utilizing Java technology.

- Instrumentation level — gives instant manageability to *any* Java technology-based object. This level is aimed at the entire developer community utilizing Java technology. This level provides *management of Java technology* which is standard across all industries.
- Agent level — provides management agents. JMX agents are containers that provide core management services which can be dynamically extended by adding JMX resources. This level is aimed at the management solutions development community and provides *management through Java technology*.
- Manager level — provides management components that can operate as a manager or agent for distribution and consolidation of management services. This level is aimed at the management solutions development community and completes the *management through Java technology* provided by the Agent level.

In order to build upon existing management technologies, the JMX specification also provides interfaces to the most widespread protocols in use today:

- Additional management protocol APIs provide a means of interacting with other management environments. The Additional management protocol APIs are aimed at the management systems development community and provide *integration with existing management solutions*.

JMX Components

The following section provides a brief introduction to JMX terminology. The JMX specification provides compatibility test suites which confirm that specific components comply fully with the appropriate section of the JMX specification.

JMX Manageable Resource

A *JMX manageable resource* is a resource that has been instrumented in accordance with the JMX Instrumentation Level Specification and tested against the Instrumentation Level Compatibility Test Suite.

A resource can be a business application, a device, or the software implementation of a service or policy. In order to be instrumented, a resource can be fully written in the Java programming language or just offer a Java technology-based wrapper. Anything that needs to be managed, now or in the future can be instrumented and considered as a potential resource.

A managed bean, or *MBean* for short, is a Java object that represents a JMX manageable resource. By design, MBeans also follow the JavaBeans™ components model, thus providing a direct mapping between JavaBeans components and manageability. Because MBeans provide instrumentation of managed resources in a standardized way, they can be plugged into any JMX agent.

JMX Agent

A *JMX agent* is a management entity implemented in accordance with the JMX Agent Specification and tested against the Agent Level Compatibility Test Suite. A JMX Agent is composed of an MBean server, a set of MBeans representing managed resources, and at least one protocol adaptor or connector. A JMX Agent may also contain management services, also represented as MBeans.

The *MBean server* is a registry for MBeans in the agent. The MBean server is the component which provides the services allowing the manipulation of MBeans. All management operations performed on the MBeans are done through Java technology-based interfaces on the MBean server.

Protocol adaptors and *connectors* let management applications access a JMX agent and manipulate the MBeans it contains. Protocol adaptors give a representation of the MBeans directly in another protocol, such as HTML or SNMP. Connectors include a remote component that provides end-to-end communications with the agent over a variety of protocols (for example HTTP, HTTPS, IIOP). Since all connectors have the same Java technology-based interface, management applications use the connector most suited to their networking environment and even change connectors transparently as needs evolve.

JMX Manager

A JMX manager is a management entity implemented in accordance with the JMX Manager Specification and tested against the Manager Level Compatibility Test Suite. A JMX manager provides an interface for management applications to interact with the agent, distribute or consolidate management information, and provide security. JMX managers can control any number of agents, thereby simplifying highly distributed and complex management structures.

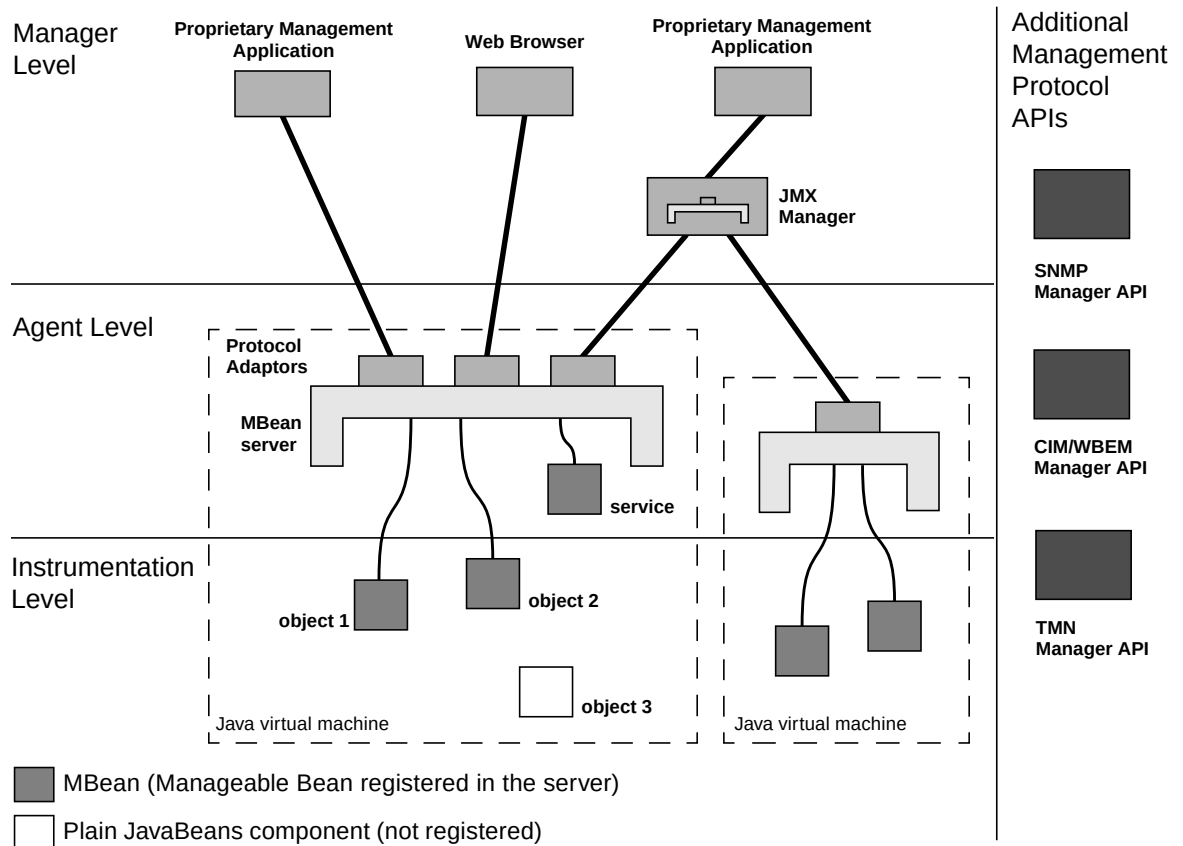


FIGURE 1 Key Components of the Java Management extensions

Services for Management

Both JMX agents and JMX managers integrate services that give them autonomy and intelligence. These services enable agents to handle their resources and let managers forward information back and forth between agents and management applications. Agents are more autonomous because they can incorporate certain management tasks, such as polling. The intelligence is embodied in simple logic that can keep managers from escalating unimportant alarms. Both of these measures can reduce network traffic and make management applications more resistant to outages.

In the JMX architecture, services are also MBeans that can be added and removed as needs evolve. This gives scalability to agents and managers, which is critical when these are deployed on thin clients.

The JMX specification currently defines the interface for such basic services as a registry for MBeans, queries of this registry, operations on resources and the forwarding of events back to managers, dynamic loading of new MBeans, creation of relationships and dependencies between MBeans, timer functions and attribute monitoring. Other management services that will be integrated into the specification include bootstrapping and persistence, network policy management, discovery of agents and managers, and security.

Additional Management Protocol APIs

The goal of these APIs is to provide a standard way for Java management applications to interact with existing management technologies. Typically, an application will use one of these APIs to access a legacy system and expose its attributes as a JMX manageable resource. This resource will then allow any JMX-compliant management application to manage the legacy system through a JMX agent. These APIs therefore create a bridge between existing and future technologies.

Based on the experience and feedback of the community utilizing Java technology, the JMX specification will include an SNMP Manager API, WBEM Manager and Provider APIs, and TMN Manager, Alarm and Topology APIs.

A New Management Paradigm

Why Java Technology is a Must for the Next Generation of Management Systems

The service driven network is taking over from the client-server model as it becomes incapable of satisfying the demands of today's customers. That is, the world is moving into the service age. Only a service-centered, service-driven network can satisfy these demands. Such a network can only exist if it is manageable, and the management system must be as flexible as the services it is managing.

JMX provides the flexibility, inter-operability, and dynamic management capabilities that are required for the service-driven network.

Management of Java Technology

The JMX instrumentation level delivers a light-weight instrumentation technology that surpasses any existing management technology in terms of flexibility. JMX provides instrumentation and hence manageability to any Java technology-based object, in all areas of the industry. JMX instrumentation allows any Java technology-based resource to be spontaneously managed by any JMX compliant agent.

Using JMX leaves application developers free to put the effort into their core business, rather than putting excess effort into adding manageability. Often, adding a few lines of Java technology-based code is enough to have instant manageability. This provides Java application developers with a standard way of instrumenting Java technology-based code. Developers writing code not based on the Java programming language can also add manageability by adding a Java technology-based wrapper which includes the instrumentation for their application. Code instrumented in this manner gains manageability that is not tied to any protocol or information model.

JMX MBean technology gives:

- Standard manageability for any Java application, sometimes in just three to five additional lines of code
- The ability to embed all necessary management information in a standard way in the resource to be managed
- The ability to provide a wrapper for instrumented resources not based on Java technology (even proprietary or custom solutions) with Java technology-based management systems

A JMX compliant agent is automatically capable of managing JMX resources. A non-JMX agent may also support JMX resources. Finally, the JMX instrumentation layer does not introduce any dependencies on external classes, a resource is entirely self-contained.

JMX instrumentation is aimed at the *entire developer community that utilizes Java technology*. JMX meets the demands for easy, rapid instrumentation of Java code allowing any resource to be spontaneously and dynamically managed.

Management *through* Java technology

The JMX agent and manager levels provide a flexible, distributed, and dynamic management infrastructure to be implemented in the Java programming language. By extending the Java programming language, JMX enhances the capabilities of existing solutions and enables the rapid creation and deployment of new types of management solutions. These solutions can be extended dynamically to incorporate new equipment and services in a plug and play manner.

JMX agents and managers developed using Java technology offer the following benefits:

- Platform independence
- Protocol independence
- Information model independence
- Management application independence

The JMX agent and manager levels are tailored for use by the following segments of the management solutions developer community:

- Management application suppliers
- Management platform vendors
- Equipment and device manufacturers
- System integrators

Each of these groups should consider the unique benefits that Java technology brings to management solutions. If they already use Java technology, JMX is the specification they should rely on to achieve standard Java technology-based management.

This approach uses the best of Java technology to create a new generation of management systems. JMX gives developers the freedom to integrate with existing and future Java technology in any field. For example, Jini™ technology-enabled devices are already manageable using JMX technology.

Integration with Existing Management Solutions

The JMX additional management protocol APIs bring the breakthrough technology of JMX into the reach of any existing management solution. The JMX technology though, does not attempt to replace existing legacy systems. This is simply not an option for end-users who must consider the value of the investment they have made. Introducing new technology beside an existing management system is also not a viable option, as this would create two separate management entities unable to cooperate or communicate effectively.

Instead, JMX offers a vertical integration, either providing manager or agent services through the technologies already in place. JMX provides a means for the seamless introduction of the latest Java technologies into existing management systems.

JMX includes an open interface that any management system vendor can leverage. Using this interface, a JMX agent and its resources can present management information consistent with various management models, such as:

- SNMP
- CIM / WBEM
- CORBA

- TMN
- LDAP

The JMX specification includes the definition of several management protocol APIs. These allow JMX managers to access agents in a legacy system and communicate with them through an existing protocol. For example, the SNMP manager API provides the services needed to write applications that manage SNMP agents or act as SNMP proxies. The definition of a WBEM client API allows you to write Java applications that access a CIM Object Manager.

Advantages of JMX

JMX is a unique management specification because it encompasses more than the specification of an interface. JMX also provides developers with the following:

- The shared expertise gained through the Java Community Process
- A complete set of tools for developers
- A proven technology base

Java Community Process

The JMX specification has been developed according to the Java Community Process (JCP) for defining Java extensions. The JCP represents the formalization of the open process that Sun Microsystems, Inc. has been using since 1995 to develop and revise Java technology specifications in cooperation with the international community utilizing Java technology.

By following the JCP, the development of the JMX specification follows a transparent process that involves the whole Java community. This insures the widest experience base for contributions and insures as much as possible that the specification is aligned with real business needs in the Java technology-based and management communities.

Complete Set of Tools for Developers

JMX is not only a specification, but is a complete set of “tools” for the developer. Standards are generally specification documents that only describe interfaces to technologies, and the developer is faced with the task of implementing conformant applications from scratch. This difficulty is compounded by the lack of a test suite for validating conformance.

In this respect, JMX is much more than a specification. JMX is supplied as a package containing the following components:

- Specification
- APIs
- Javadoc™ reference information
- Reference Implementation
- Compatibility Test Suite

As a result, developers utilizing Java technology can use the JMX package in all of their product phases, from design and prototyping, through development and testing. This will help the community utilizing Java technology bring JMX-compliant products to market much faster, and this for all levels of the specification.

Compatibility Test Suite

The Compatibility Test Suite (CTS) checks that JMX implementations actually conform to the specification. JMX services are divided into mandatory services and optional services. A JMX-compliant implementation provides all mandatory services and any subset of the optional services. It may also provide other services which are not in the specification.

Reference Implementation

The JMX specification provides a Reference Implementation (RI) of JMX services. The RI contains Java technology-based classes for all specified objects, and it is an implementation which passes the CTS. The RI allows developers of JMX-based management solutions to prototype their applications easily. It also provides a working model for developers of the infrastructure. This is especially useful for those providing additional services for JMX agents or managers.

Proven Technology Base

JMX is a new paradigm for management, however, it is based on proven technology. The concepts of Java management have already been deployed in the Java Dynamic Management™ Kit, a product which has been shipping for two years.

This implementation has shown that the instrumentation of Java technology-based code is indeed very simple, that resources can be plugged in and out of agents dynamically, that management services bring autonomy to managers, and that the Java programming language can interface effectively with an existing protocol such as SNMP.

Conclusion

JMX brings manageability to Java technology across all industries, providing a lightweight, standard way to instrument any Java technology-based application, service, or device. As a developer utilizing Java technology, you will find that your Java technology-based code can become universally and dynamically managed, quickly and with little effort.

JMX brings the best of Java technology to management, making a new generation of flexible, spontaneous management systems possible. As a developer of system and network management solutions, you will find that you are able to create and deploy new solutions that can be extended dynamically to incorporate new devices and services.

JMX integrates simply with existing management technologies such as SNMP, future technologies such as CIM/WBEM, as well as proprietary technologies. As a system integrator, you will find that you are able to seamlessly introduce the latest Java technologies into existing management systems.

The new management paradigm for the service-driven networks that JMX brings to the industry is based on proven concepts and an architecture already used in the Java Dynamic Management Kit.

Dynamic Management for the Service Age

In the dawning Service Age, the client-server computing model will soon be replaced by a device- and service-driven network and data-center model. Devices, services and data-centers will all change very rapidly under market pressure. New devices will be introduced daily, content owners will bring new streams of information, and service providers will add customized services daily. Networks and data-centers will be permanently available, like the phone networks today.

JMX brings the standardized management infrastructure and provides the tools needed to keep control of this fast moving environment and to take advantage of it for faster, more efficient service delivery.



THE NETWORK IS THE COMPUTER™

Sun Microsystems, Inc.
901 San Antonio Road
Palo Alto, CA 94303
650 960-1300

For U.S. Sales Office locations, call:
800 821-4643
In California:
800 821-4642

Australia: (02) 413 2666
Belgium: 32 2 716 7911
Canada: 416 477-6745
Finland: 358-0-502 27 00
France: (1) 30 67 50 00
Germany: (0) 89-46 00 8-0
Hong Kong: 852 802 4188
Italy: 039 60551
Japan: (03) 221-7021
Korea: 822-563-8700
Latin America: 415 688-9464
The Netherlands: 033 501234
New Zealand: (04) 499 2344
Nordic Countries: +46 (0) 8 623 90 00
PRC: 861-831-5568
Singapore: 224 3388
Spain: (91) 5551648
Switzerland: (1) 825 71 11
Taiwan: 2-514-0567
UK: 0276 20444

Elsewhere in the world,
call Corporate Headquarters:
650 960-1300
Intercontinental Sales: 650 688-9000